CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-[Section]
System Integration and Architecture

System Design Document (SDD)

Project Title: GuildHall

Prepared By: Leanda, John Luis Catarina

Version: 3.1

Date: 4/15/2026
Status: Review

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1.0	2/18/2026	John Luis Leanda	Initial draft	Draft
2.0	3/3/2026	John Luis Leanda	All sections completed	Review
2.1	3/4/2026	John Luis Leanda	Updated database to Supabase	Review
3.0	4/12/2026	John Luis Leanda	New api endpoints, updated diagrams and detailed feature explanations	Review
3.1	4/15/2026	John Luis Leanda	Updated chat system	Review
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY..... 4

1.0 INTRODUCTION..... 5

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION..... 5

3.0 NON-FUNCTIONAL REQUIREMENTS..... 9

4.0 SYSTEM ARCHITECTURE..... 11

5.0 API CONTRACT & COMMUNICATION..... 12

6.0 DATABASE DESIGN..... 17

7.0 UI/UX DESIGN..... 18

8.0 PLAN..... 21

EXECUTIVE SUMMARY

1.1 Project Overview

GuildHall is a gamified, community service platform designed to facilitate local volunteering and mutual aid through an "Adventurer's Guild" theme. Users can post localized "Quests" (tasks or requests for help) and earn Experience Points (XP) upon completion. The ecosystem consists of a Spring Boot REST API, a Supabase (PostgreSQL) relational database, a React Vite web dashboard for community management, and a Kotlin Android application.

1.2 Objectives

1. Develop a functional community service platform with JWT authentication, quest creation, quest catalog, and quest acceptance logic.
2. Implement a leveling and XP system to encourage user engagement and local volunteering. Rank titles (e.g., Bronze, Silver, Gold, Mithril, Adamantite) are assigned based on level thresholds defined in the backend.
3. Ensure seamless data synchronization between the Android mobile client (for field use) and the React web client (for community management).
4. Allow admins to create "Guilds" which are dedicated groups separating different users. Users can then join these groups.
5. Develop a centralized Spring Boot backend that handles complex business logic, such as quest state transitions and user progression.
6. Implement JWT-based authentication to protect user data and ensure quest integrity across all integrated systems, including backend-driven Google OAuth2.
7. Leverage Supabase Storage for quest briefing file attachments (images and PDFs), with public URLs persisted in the database.

1.3 Scope

Included Features:

- User or "Adventurer" profiles featuring levels, XP bars, and "Rank" tags (Bronze → Silver → Gold → Mithril → Adamantite based on level).
- Secure login using JWT and BCrypt hashing. Support for native user registration and backend-driven Google OAuth2 (server-side authorization code flow).

- Quest Board (CRUD): Creating, viewing, and deleting quests. Quest acceptance logic is implemented and pending merge to main branch.
- File attachment support via Supabase Storage: images compressed client-side before upload; PDFs uploaded directly. Public URLs stored in the quests table.
- Grouping users and quests into communities or "Guilds" based on user choice.
- Admin or "Guildmaster" panel for global guild management, guild creation, guild renaming, guild deletion, user viewing, and user banning.
- Real-time daily "Heroes' Wisdom" quote fetched from the Quotable API (server-side proxy with in-memory daily caching).
- Responsive React web interface with a mobile-first Kotlin Android application.
- Supabase (PostgreSQL) schema to handle complex relationships between users, guilds, quests, payments, messages, and memberships.
- Profile management: bio editing, skill selection, and profile picture upload (stored as base64 or URL via the /profile/me endpoint).
- Auto-enrollment of new users into the "Global Square" default guild on registration.

Excluded Features:

- Logistics or delivery of physical items
- Video chat
- Public global chat
- Map API integration

1.0 INTRODUCTION

1.1 Purpose

This document serves as the formal design specification for GuildHall, a gamified community service platform. It outlines the architectural framework, technical requirements, and integration strategies necessary to connect a Spring Boot backend, a React web frontend, and a Kotlin Android mobile application. This document ensures that the "Adventurer Guild" logic features are consistently implemented across all system tiers.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: GuildHall

Domain: Community Service / Hyper-local Volunteering

Primary Users: Small organizations like schools, Barangay communities, or freelancers

Problem Statement: Smaller communities in the Philippines often lack a centralized, engaging way to request small favors or find volunteering opportunities. Existing social media groups like Facebook groups are cluttered and lack accountability or incentive systems.
Solution: A themed "Adventurer's Guild" platform that gamifies real-world help through a Quest/XP system, providing a structured environment for community cooperation with dedicated mobile and web interfaces.

2.2 Core User Journeys

Journey 1: New Adventurer Registration & Onboarding

1. User visits the Kotlin Mobile App or React Web App.
2. User selects "Sign Up" (native form) or "Continue with Google" (backend-driven OAuth2 flow).
3. For Google OAuth: the browser is redirected to GET /api/v1/auth/google/init. The backend redirects to Google's consent screen, then handles the callback at /api/v1/auth/google/callback entirely server-side, and finally redirects the browser to /auth/google/success?token=<JWT>&newUser=true|false.
4. System verifies credentials, creates a new record in the users table, and issues a GuildHall JWT.
5. System auto-enrolls the new user in the "Global Square" default guild.
6. User is redirected to the Skills Selection screen (if new user) or the Guilds dashboard.
7. User views their profile, initialized at Level 1 with 0 XP.

Journey 2: Posting a Quest

1. Poster (User) fills out the "Commission Quest" form: title, category, description, reward type, and optional file attachment.

2. If a file is attached: images are compressed client-side (max 800px, JPEG 70%) before being uploaded to Supabase Storage. PDFs are uploaded directly (max 500 KB). The public URL is stored in the quest record.
3. Poster sets the reward type to either "volunteer" or "paid". If paid, enters the reward amount.
4. Quest is saved with status OPEN and appears immediately on the guild's quest board.

Journey 3: Completing a Quest

1. Helper (User) accepts an OPEN quest. Its status changes to PENDING.
2. Helper (User) accomplishes the task. They can chat with the Poster when necessary for more details.
3. The Poster clicks "Confirm Completion."
4. System Logic Triggers:
    - Payment is finalized.
    - XP is automatically awarded to the Helper.
    - SMTP Email is sent to the Helper: "Your quest was confirmed! +(amount) XP."

Journey 4: Guildmaster Validation & Reward

1. Guildmaster (Admin) logs into the React Dashboard.
2. They see a global log of all quests to ensure no "illegal" or "scam" quests are posted.
3. Guildmaster has the power to "Flag" or "Delete" quests that violate community standards.
4. Dispute Resolution: If a Helper claims they finished but the Poster refuses to pay, the Guildmaster reviews the provided proof from either chat and can manually trigger the completion.

2.3 Feature List (MoSCoW)

MUST HAVE

1. Authentication & Security: User registration/login with JWT (24-hour expiration), BCrypt password hashing (12 rounds), and backend-driven Google OAuth2 (server-side authorization code exchange).

2. Quest Board (Core CRUD): Full Create, Read, and Delete capabilities for Quests scoped within a guild. Update is scaffolded.
3. File Attachment: Upload of image or PDF to Supabase Storage. Images are compressed client-side before upload.
4. Guild Management: Users can join and leave guilds. Admins can create, rename, and delete guilds.
5. User Profile: Bio editing, skill selection, profile picture upload, XP bar, and rank display.
6. Heroes' Wisdom: Backend proxy to the Quotable API with server-side daily caching (sessionStorage on frontend).
7. Role-Based Access Control: Distinct views and API restrictions for Adventurers (ROLE_ADVENTURER) and Guildmasters (ROLE_GUILDMASTER).
8. Mobile & Web Frontends: Responsive React Vite site and Kotlin Android app.
9. Payment Gateway for paid quests.

SHOULD HAVE

1. Quest-Linked Chat: Real-time messaging between Poster and Helper.
2. XP & Leveling System: Backend logic to calculate user rank based on completed quest difficulty.
3. Search & Filtering: Ability to filter quests by category (e.g., Tutoring, Manual Labor).
4. User Profile Dashboard: A view for users to see their current Level, XP progress, rank, and skills.

COULD HAVE

1. Quest Leaderboard: A global ranking of top adventurers in specific communities
2. Achievement Badges: Special icons for completing specific types of quests (e.g., "Scholar" for 5 tutoring quests).
3. Dark Mode: Themed UI toggle for both Web and Mobile.
4. Push Notifications: Sending alerts to the Android app when a nearby quest is posted.

WON'T HAVE

1. Real Money Transactions: Only Sandbox/Test mode will be implemented.
2. Advanced Analytics: Complex data visualization for "Grandmasters" is deferred.
3. Map API integration.
4. Multi-language Support: The initial system will be English-only.

2.4 Detailed Feature Specifications

Feature: Authentication & Identity

- **Screens**: Registration, Login, Skills Selection, Google Callback, Profile View.
- **Security**: BCrypt hashing (12 rounds), JWT stateless tokens (24-hour expiry), backend-driven Google OAuth2 (authorization code → token exchange server-side; frontend never touches Google APIs directly).
- **Session Management**: JWT stored in localStorage (key: guildhall_token) on web; SharedPreferences on Android.
- **API Endpoints**:
    - POST /api/v1/auth/register (native signup; auto-enrolls in Global Square)
    - POST /api/v1/auth/login (accepts username OR email in the username field)
    - GET /api/v1/auth/google/init (redirects browser to Google consent screen)
    - GET /api/v1/auth/google/callback (handles Google redirect; issues JWT; redirects to /auth/google/success)
    - POST /api/v1/auth/skills (saves skill selections post-onboarding)
    - GET /api/v1/auth/me (returns full current user context)
    - POST /api/v1/auth/logout (stateless; clears client-side token)
    - PUT /api/v1/profile/me (updates bio and/or profilePictureUrl)

Feature: The Quest Board (Core CRUD)

- **Screens**: Guild Dashboard (Quest Board), Commission Quest Modal, Quest Detail Modal.
- **File Attachments**: Images compressed client-side (canvas, max 800px, JPEG 70%), uploaded to Supabase Storage bucket quest-attachments at path guild-{guildId}/{timestamp}-{random}.{ext}. Public URL stored in quests.attachment_path. PDF files capped at 500 KB and also stored in Supabase Storage.
- **Image Lightbox**: Full-screen image viewer with zoom (scroll/+/-), pan (drag), and reset controls.
- **Quote Integration**: GET /api/v1/wisdom fetches a random daily quote. Backend caches the quote in-memory (AtomicReference) per calendar day, falling back to hardcoded quotes if Quotable is unavailable.
- **API Endpoints**:
    - GET /api/v1/guilds/{guildId}/quests (member-only; returns OPEN quests)
    - POST /api/v1/guilds/{guildId}/quests (creates quest; JSON body includes attachmentName and attachmentPath URL)
    - GET /api/v1/guilds/{guildId}/quests/{questId} (view specific quest)
    - DELETE /api/v1/guilds/{guildId}/quests/{questId} (poster or Guildmaster only)
    - GET /api/v1/quests/mine (returns all OPEN quests posted by the current user across all guilds)
    - GET /api/v1/wisdom (public; backend proxy to Quotable API)

Feature: Payments (Reward System)

- **Screens:** Checkout, Payment Success/Failure.
- **Integration:** Stripe Sandbox (Test Mode).
- **Persistence:** Every quest is linked to a transaction record in the payments table.

- **API Endpoints:**
    - POST /api/payments/create-session/{questId} (Initiates Stripe Checkout, if quest type is paid)
    - GET /api/payments/verify/{sessionId} (Updates payment status in database)

Feature: Adventurer Chat (Real-Time)

- **Screens:** Chat Sidebar (list of conversations), Conversation View (message history)
- **Functions:** Private 1-to-1 text communication between any two users. Automated SYSTEM notifications sent when a quest is accepted.
- **Implementation:** WebSocket (STOMP) for live updates; REST fallback for message history and system notifications.
- **API Endpoints:**
    - WS /app/chat/{conversationId} (WebSocket connection point for sending messages)
    - GET /api/v1/chat/conversations (Fetch all conversations for current user)
    - POST /api/v1/chat/conversations/{otherUserId} (Open or create conversation with another user)
    - GET /api/v1/chat/conversations/{conversationId}/messages (Fetch message history for a conversation)
    - POST /api/v1/chat/conversations/{conversationId}/messages (REST fallback to send message or SYSTEM notification)
    - GET /api/v1/chat/users/search?q= (Search for users to start conversations)
    - POST /api/v1/chat/system/quest-accepted (Internal: triggered when quest is accepted)

Feature: Guildmaster Global Control (Admin)

- **Access Control:** Global RBAC. Requires ROLE_GUILDMASTER, enforced at SecurityConfig level via requestMatchers("/api/v1/admin/**").
- **Functions**:
    - Guild Oversight: GET, POST (create), PUT (rename), DELETE all guilds.
    - User Management: GET all users, GET single user profile, POST ban (deletes user and their memberships).
- **API Endpoints**:
    - GET /api/v1/admin/guilds
    - POST /api/v1/admin/guilds
    - PUT /api/v1/admin/guilds/{id}
    - DELETE /api/v1/admin/guilds/{id}
    - GET /api/v1/admin/users
    - GET /api/v1/admin/users/{id}
    - POST /api/v1/admin/users/{id}/ban

2.5 Acceptance Criteria

AC-1: User Onboarding & Auto-Enrollment

Given I am a new user registering for the first time,

When my account is successfully created in the database,

Then the system must automatically assign me an ACTIVE membership to the "Global Square" guild,

And I must be directed to the Skills Selection screen before reaching the guild list.

AC-2: Integrated User Onboarding (Security & OAuth)

Given I am a new user on the Web or Android app,

When I register via a native form or Google OAuth,

Then the system must hash my password using BCrypt (if native) and save my record in the users table,

And the system must return a valid JWT to my device.

For Google OAuth: the entire token exchange must happen server-side with no Google SDK calls from the frontend.

AC-3: Quest Commissioning (Paid)

Given I am a logged-in Adventurer (User) within a specific Guild,

When I create a quest and optionally upload a Briefing File (PDF/Image),

And I set the quest type to “paid”,

And I complete the Stripe Sandbox payment for the "reward,"

Then the quest must appear on that Guild’s board with the attachment accessible for download,

And a record must be created in the payments table linked to the quest.

AC-4: Quest Commissioning (Volunteer)

Given I am a logged-in Adventurer (User) within a specific Guild,

When I create a quest and optionally upload a Briefing File (PDF/Image),

And I set the quest type to “volunteer”,

Then the quest must appear on that Guild’s board with the attachment accessible for download,

AC-5: Quest Deletion

Given I am the poster of a quest or a Guildmaster,

When I delete the quest,

Then the quest must be removed from the board and the database immediately.

AC-6: Community Interaction (User Chat)

Given I am a logged-in Adventurer,

When I open a conversation with another user or when a quest is accepted,

Then I can see the conversation list on the Chat sidebar,

And I can send and receive real-time text messages via WebSocket,

And I can receive automated SYSTEM notifications (e.g., "{Username} accepted your quest"),

And each message must be persisted in the direct_messages table linked to the conversation.

AC-7: Validation & Rewards (Logic & RBAC)

Given I am the poster of a quest or a Guildmaster (Admin),

When I review the completed work and click "Approve,"

Then the system must update the Helper's XP and Level in the users table,

And an SMTP Email must be sent to the Helper confirming their reward,

And the quest must transition to the COMPLETED state in the database.

AC-8: Admin Guild Operations

Given I am logged in as the Guildmaster,

When I create, rename, or delete a guild from the Admin Dashboard,

Then the change must be reflected immediately in the guild list without a page refresh,

And deleting a guild must also remove all associated memberships.

AC-7: Global Moderation (RBAC)

Given I am logged in as the Guildmaster,

When I identify a quest that violates the terms of service,

Then I must be able to delete that quest immediately via the DELETE /api/v1/admin/quests/{id} endpoint.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API Response Time: ≤ 2 seconds for 95% of standard REST requests.
- External API Latency: The "Heroes' Wisdom" (Quotable API) must fetch and render within 1.5 seconds of page load.
- File Upload/Download Latency: Quest briefings (PDF/Images up to 10MB) must finish uploading within 8 seconds on standard 4G connections.
- Concurrent Users: Support 100 concurrent adventurers interacting across different Guilds.
- Real-time Latency: Chat messages must be delivered within 1 second using WebSockets (STOMP).

3.2 Security Requirements

- Encryption: HTTPS/TLS 1.3 for all data in transit.
- Identity: JWT-based stateless authentication with a 24-hour expiration.
- Social Auth: Secure handling of Google OAuth 2.0 tokens; internal IDs must be linked to Google sub (subject) IDs.
- Data Protection: Password hashing using BCrypt with 12 salt rounds.
- Scoped RBAC: Server-side verification of roles (ROLE_ADVENTURER, ROLE_GUILDMASTER) before accessing restricted endpoints.

3.3 Compatibility Requirements

- Web Frontend (React Vite): Chrome, Firefox, Safari, and Edge. Optimized for Guildmasters to manage guilds and oversee the system.
- Mobile Frontend (Kotlin): Android API Level 24+ (Android 7.0 Nougat; minSdk = 24, targetSdk = 34).
- Responsive Design: Fully responsive from 320px (mobile) to 2560px (desktop). Mobile-first approach with breakpoints at 640px, 768px, 1024px.

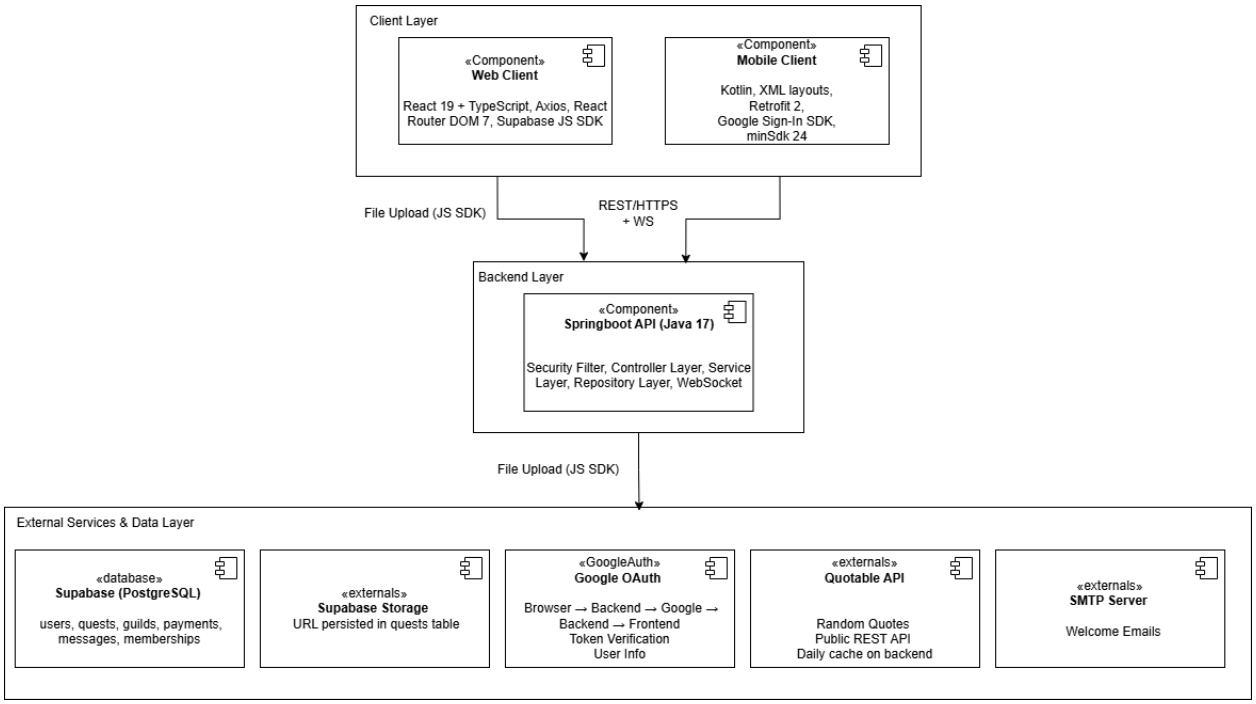
3.4 Usability Requirements

- Onboarding Speed: New users should be able to "Continue with Google" and join their first Guild within 2 minutes.
- Accessibility: Following WCAG 2.1 guidelines.

- Thematic Consistency: Error messages and success alerts should use "Guild" terminology (e.g., "Quest Commission Failed" instead of "Form Submission Error").
- Mobile-First Design: Minimum 48x48px touch targets for core actions like "Accept Quest" or "Open Briefing."

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 19, Spring Boot 3.5.11, Spring Security (JWT), Spring Data JPA, Spring WebSocket (STOMP), Spring Mail
- **Database:** Supabase (PostgreSQL); Supabase Storage for file attachments; Supabase JS SDK (@supabase/supabase-js ^2.100.1) on the web frontend
- **Web Frontend:** React 19 (Vite 7), TypeScript 5.9, Axios 1.x, React Router DOM 7.x, @react-oauth/google 0.13.x (used only for Android native flow; web uses backend-driven redirect)
- **Mobile:** Kotlin, XML-based layouts, Retrofit 2 (OkHttp3), Google Sign-In SDK (play-services-auth 21.0.0); minSdk 24, targetSdk 34

- **Build Tools:** Maven 3.9 (Backend), npm / Vite (Web), Gradle 8.4 / Kotlin 1.9 (Android)
- **External APIs:** Quotable API (api.quotable.io) — random inspirational quotes; Google OAuth2 (accounts.google.com)
- **Deployment:** Render (Web / Backend); APK sideload (Mobile); Supabase cloud (Database & Storage)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

- **Base URL:** https://api.guildhall.app/api/v1
- **Format:** JSON for all requests/responses
- **Authentication:** Bearer token (JWT) in Authorization header
- **WebSocket Connection:** wss://api.guildhall.app/ws (STOMP; scaffolded)

Standard Response Envelope:

```
{
  "success": boolean,
  "data": object | null,
  "error": {
    "code": string,
    "message": string,
    "details": object | null
  },
  "timestamp": string   // ISO-8601, e.g. "2026-03-01T10:30:00Z"
}
```

5.2 Endpoint Specifications

Authentication Endpoints

Method	Path	Body / Params	Response / Notes
POST	/auth/register	{email, username, password}	{user, token}; auto-enrolls in Global Square; isNewUser=true
POST	/auth/login	{username, password} (username field accepts email too)	{user, token}
GET	/auth/google/init	None (browser redirect)	Redirects browser to Google consent; no body returned
GET	/auth/google/callback	?code=... (from Google)	Exchanges code server-side; redirects to /auth/google/success?token=<JWT>&newUser=<bool>
POST	/auth/skills	{skills: string[]}	{user, token}; authenticated
GET	/auth/me	Bearer token	{user: {id, email, username, role, level, xp, rank, skills, bio, profilePictureUrl, googleSub}}
POST	/auth/logout	Bearer token	{message}; stateless — client must discard JWT
PUT	/profile/me	{bio?, profilePictureUrl?}	{bio, profilePictureUrl}; authenticated

Guild Endpoints

Method	Path	Auth	Response / Notes
GET	/guilds	Authenticated	{guilds: [..., isMember: bool]}; all guilds
GET	/guilds/my	Authenticated	{guilds: [...]}; only user's joined guilds
GET	/guilds/{id}	Member only	{guild: {...}}; 403 if not member

Method	Path	Auth	Response / Notes
POST	/guilds/{id}/join	Authenticated	{message, guild}; 409 if already member
DELETE	/guilds/{id}/leave	Authenticated	{message}; removes membership record

Quest Endpoints

Method	Path	Auth	Response / Notes
GET	/guilds/{guildId}/quests	Member only	Returns OPEN quests only. Supports keyword search server-side.
POST	/guilds/{guildId}/quests	Member only	Body: {title, category, description, questType, reward?, attachmentName?, attachmentPath?}. Status set to OPEN immediately.
GET	/guilds/{guildId}/quests/{id}	Member only	{quest: {..., attachmentName, attachmentData (URL)}}
DELETE	/guilds/{guildId}/quests/{id}	Poster or Guildmaster	{message: "Quest deleted"}
GET	/quests/mine	Authenticated	Returns OPEN quests posted by current user across all guilds, including guildId and guildName.
POST (pending)	/guilds/{guildId}/quests/{id}/accept	Authenticated member	Changes status OPEN → PENDING; records helper. Pending merge to main.
POST (pending)	/guilds/{guildId}/quests/{id}/complete	Quest poster	Changes status PENDING → COMPLETED; awards XP. Pending merge.

Wisdom / External API Endpoint

Method	Path	Auth	Notes
GET	/wisdom	Authenticated	Backend proxy to Quotable API. Same quote returned to all callers on same calendar day (in-memory AtomicReference cache). Falls back to hardcoded quote if Quotable is unavailable.

Admin / Guildmaster Endpoints

Method	Path	Auth	Notes
GET	/admin/guilds	ROLE_GUILDMASTER	All guilds with memberCount calculated from active memberships.
POST	/admin/guilds	ROLE_GUILDMASTER	Creates guild; auto-enrolls the creating Guildmaster. 409 on duplicate name.
PUT	/admin/guilds/{id}	ROLE_GUILDMASTER	Renames guild. Body: {name}.
DELETE	/admin/guilds/{id}	ROLE_GUILDMASTER	Deletes guild and all associated memberships.
GET	/admin/users	ROLE_GUILDMASTER	All users with level, xp, rank, and profilePictureUrl.
GET	/admin/users/{id}	ROLE_GUILDMASTER	Full profile of a single user including bio, skills, and googleSub.
POST	/admin/users/{id}/ban	ROLE_GUILDMASTER	Removes user's memberships then deletes user. Cannot ban another Guildmaster.
POST (scaffold)	/admin/quests/{id}/resolve	ROLE_GUILDMASTER	Manually completes a disputed quest; awards XP. Scaffolded.

User Chat Endpoints (Real-Time + REST)

Method	Path	Body/Params	Notes
WS	/app/chat/{conversationId}	WebSocket (STOMP); JSON payload: {conversationId, content}	Real-time 1-to-1 messaging; requires JWT in handshake header
GET	/api/v1/chat/conversations	Headers: Authorization: Bearer {token}	{messages: [{id, senderId, content, sentAt}]}
POST	/api/v1/chat/conversations/{otherUserId}	Authenticated	Creates or returns existing conversation; 400 if trying to message self

Method	Path	Body/Params	Notes
GET	/api/v1/chat/conversations/{conversationId}/messages	Authenticated	{messages: [{id, senderId, senderUsername, senderProfilePicture, content, messageType, questId, guildId, questTitle, guildName, isRead, sentAt}]}
POST	/api/v1/chat/conversations/{conversationId}/messages	Authenticated; Body: {content, messageType?, questId?, guildId?, questTitle?, guildName?}	REST fallback; supports TEXT and SYSTEM message types
GET	/api/v1/chat/users/search?q=	Authenticated	{results: [{id, username, profilePictureUrl, rank}]}; paginated, max 10 results
POST	/api/v1/chat/system/quest-accepted	Internal (Server-to-Server); Body: {posterId, questId, guildId, questTitle, guildName}	Auto-triggered after quest acceptance; creates SYSTEM DirectMessage

Payment Endpoints (Stripe Sandbox)

Method	Path	Body/Params	Notes
POST	/payments/create-session/{questId}	Headers: Authorization: Bearer {token}	{sessionId: string, checkoutUrl: string}
GET	/payments/verify/{sessionId}	Headers: Authorization: Bearer {token}	{payment: {id, questId, amount, status, createdAt}}
GET	/payments/history	Headers: Authorization: Bearer {token}	{payments: [{id, questId, amount, status, createdAt}]}

5.3 Error Handling

HTTP Status Codes

- 200 OK — Successful request

- 201 Created — Resource created
- 400 Bad Request — Invalid input or validation failure
- 401 Unauthorized — Authentication required or token invalid
- 403 Forbidden — Insufficient role permissions
- 404 Not Found — Resource does not exist
- 409 Conflict — Duplicate resource (e.g. username already taken)
- 422 Unprocessable Entity — Quest state transition not allowed (e.g. accepting a PENDING quest)
- 500 Internal Server Error — Unexpected server error

Error Code Examples

```
// AUTH-001: Invalid credentials
{
  "success": false,
  "data": null,
  "error": {
          "code":   "AUTH-001",
          "message": "Invalid credentials",
          "details": "Email or password is incorrect"
  },
  "timestamp": "2026-03-01T10:30:00Z"

}
```

```
// VALID-001: Validation failure
{
  "success": false,
  "data": null,
  "error": {
          "code":   "VALID-001",
          "message": "Validation failed",
          "details": {
          "title":  "Quest title is required",
          “quest_type” : “Quest type is required”,
          "reward": "(optional) Reward must be a positive number"
          }
  },
  "timestamp": "2026-03-01T10:30:00Z"

}
```

```
// QUEST-002: Invalid state transition
{
  "success": false,
  "data": null,
  "error": {
        "code":   "QUEST-002",
        "message": "Quest is not available",
        "details": "Only OPEN quests can be accepted"
  },
  "timestamp": "2026-03-01T10:30:00Z"

}
```

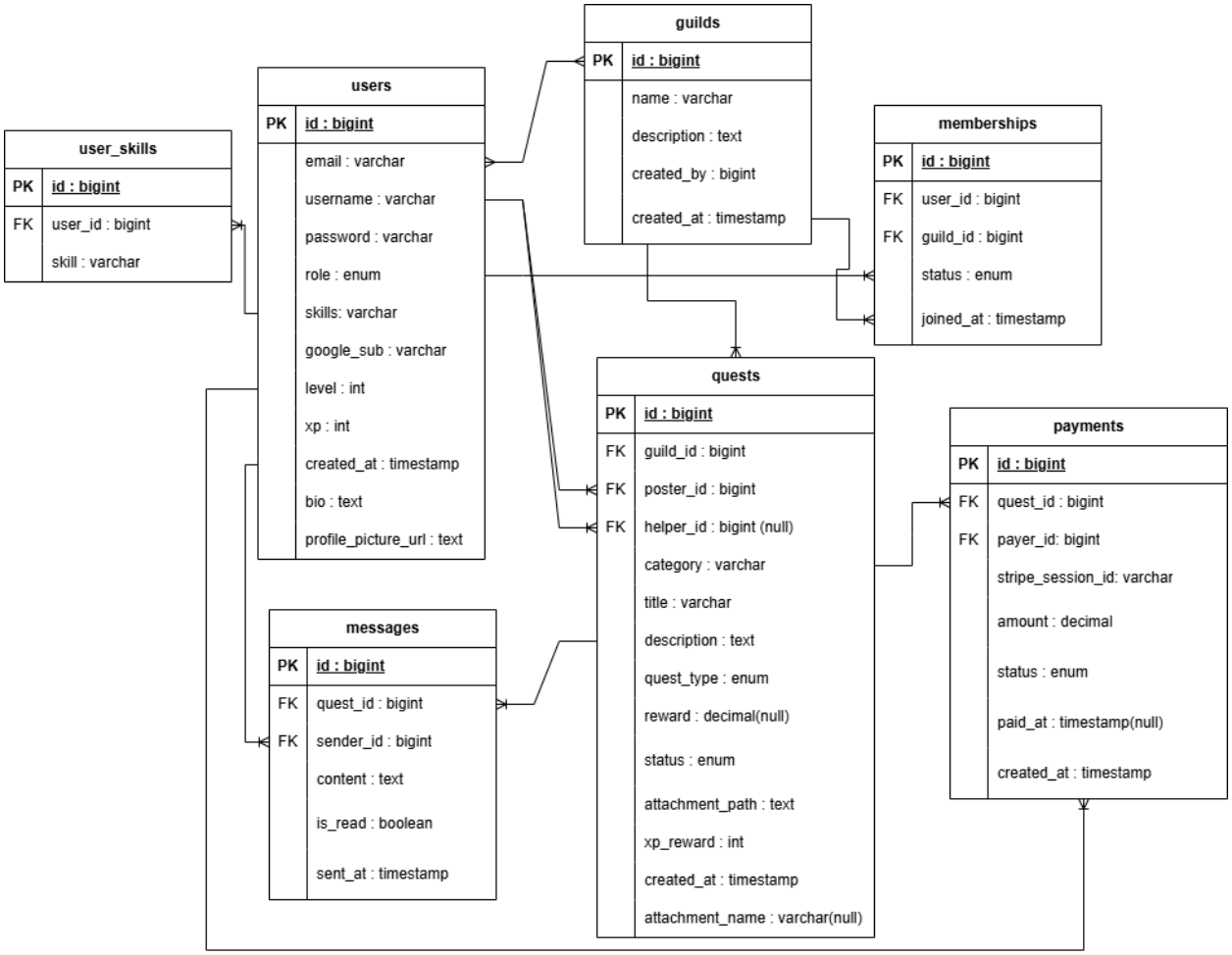
Common Error Codes

Error Code	Description
AUTH-001	Invalid email or password
AUTH-002	JWT token has expired — re-login required
AUTH-003	Insufficient permissions for this action
AUTH-004	Google OAuth token invalid or revoked
VALID-001	Request body failed validation — see details field
QUEST-001	Quest not found
QUEST-002	Invalid quest state transition (e.g. accepting a non-OPEN quest)

QUEST-003	Only the quest Poster can perform this action
GUILD-001	Guild not found
GUILD-002	User is not a member of this guild
PAYMENT-001	Stripe session creation failed
PAYMENT-002	Payment not verified — quest remains unpublished
FILE-001	File type not allowed (only PDF/JPG/PNG)
FILE-002	File exceeds 10MB size limit
DB-001	Resource not found
DB-002	Duplicate entry — resource already exists
SYSTEM-001	Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **Many-to-Many:** users ↔ guilds (resolved via the memberships bridge table)
- **One-to-Many:** guilds → quests (each guild hosts many quests)
- **One-to-Many:** users → quests as poster_id (a user can post many quests)
- **One-to-Many:** users → quests as helper_id (a user can help on many quests)
- **One-to-Many:** quests → payments (a quest has one payment record; modeled one-to-many for refund/retry scenarios)
- **One-to-Many:** users → payments as payer_id (a user can have multiple payment records)

- **One-to-Many:** users ↔ conversations (user1 and user2 fields; resolved via many-to-many Conversation entity)
- **One-to-Many:** conversations → direct_messages (a conversation contains many direct messages)
- **One-to-Many:** users → direct_messages as sender_id (a user can send many direct messages)
- **One-to-Many:** guilds → memberships (each guild has many membership records)
- **One-to-Many:** users → memberships (each user has many membership records across different guilds)
- **One-to-Many:** users → user_skill (each user can have multiple skills)

Key Tables:

1. **users —** Adventurer and Guildmaster accounts, XP, level, and authentication data
2. **guilds—** User-created groups that contain and scope all quests
3. **memberships —** Bridge table resolving the many-to-many relationship between users and guilds
4. **quests —** The core business entity; scoped to a guild with poster, helper, status, and bounty
5. **payments —** Stripe payment records linked to each quest's bounty
6. **conversations** — 1-to-1 chat channels between two users (user1_id, user2_id; unique constraint)
7. **direct_messages** — Individual messages within a conversation; supports TEXT and SYSTEM message types
8. **user_skills —** The skills that each user has selected to display in their profile.

Table Structure Summary:

- **users:** id, email, username, password, role, google_sub, level, xp, bio, profile_picture_url, created_at
- **guilds:** id, name, description, created_by, created_at
- **memberships:** id, user_id, guild_id, status, joined_at

- **quests:** id, guild_id, poster_id, helper_id, category, title, description, quest_type, reward, status, attachment_name, attachment_path, xp_reward, created_at
- **payments:** id, quest_id, payer_id, stripe_session_id, amount, status, paid_at, created_at
- **conversations:** id, user1_id, user2_id, created_at (unique constraint on user1_id, user2_id)
- **direct_messages:** id, conversation_id, sender_id, content, message_type (TEXT | SYSTEM), quest_id (nullable), guild_id (nullable), quest_title (nullable), guild_name (nullable), is_read, sent_at
- **user_skills:** user_id, skill

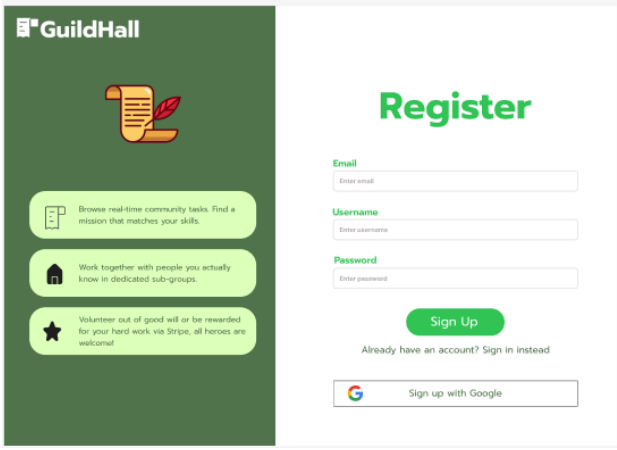
7.0 UI/UX DESIGN

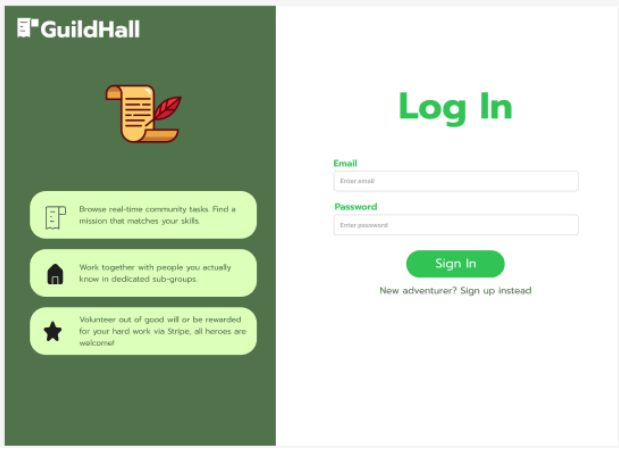
7.1 Web Application Wireframes

https://www.figma.com/design/6ax9AqEpcKSSoRqOkdJ1eq/GuildHall?node-id=0-1&t=9yxjfdkrZDXB3RWa-1

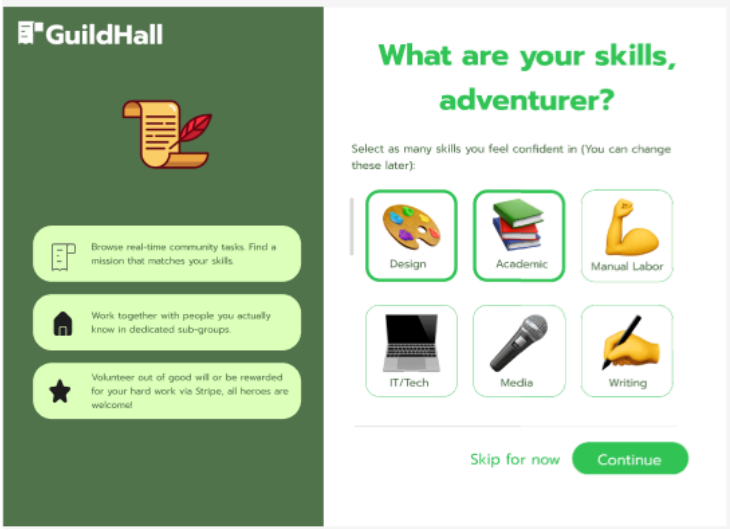
Landing Homepage

Sign Up and Sign In

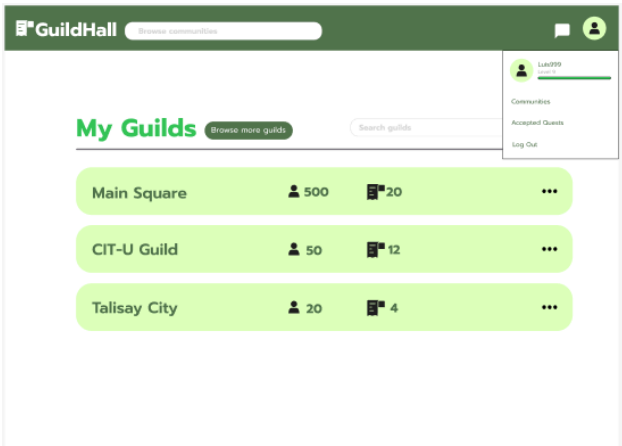


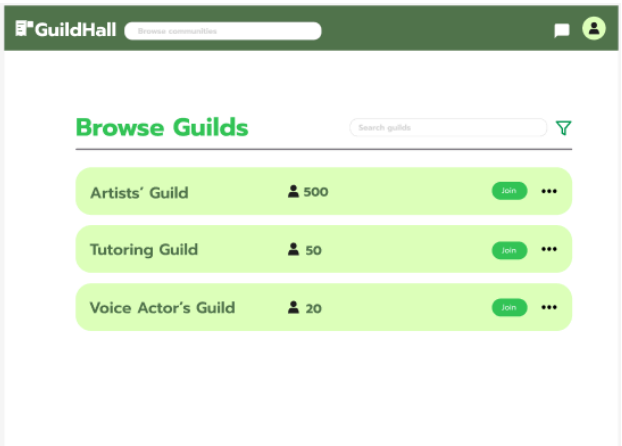


First time skills selection

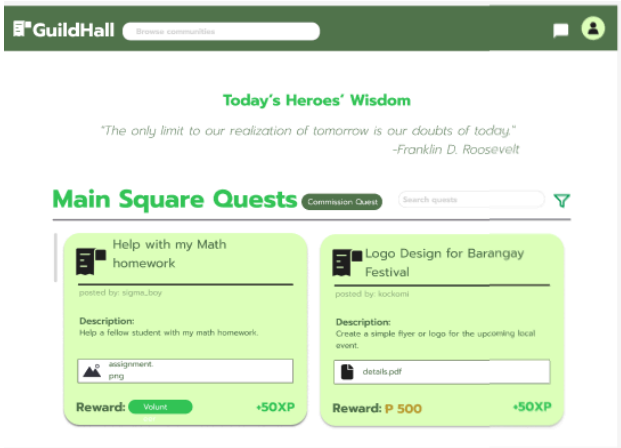


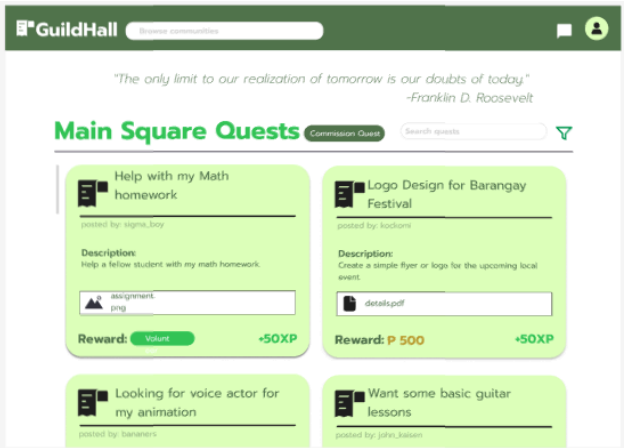
My Communities and Browse Communities



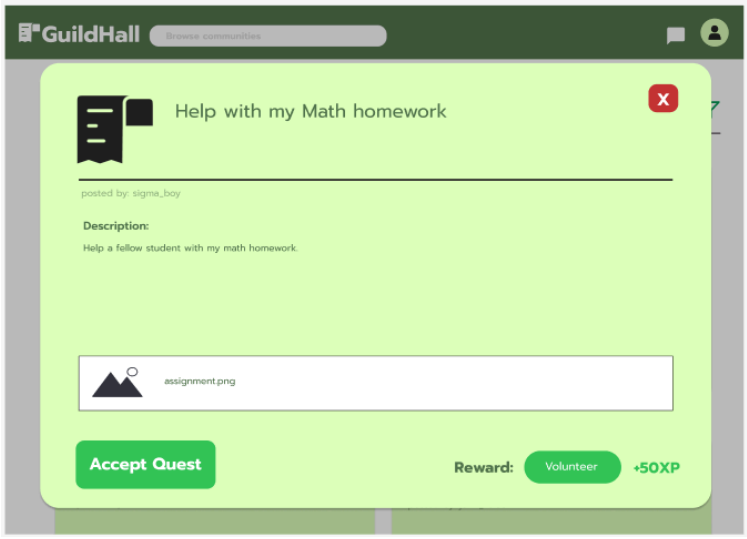


Quest Board

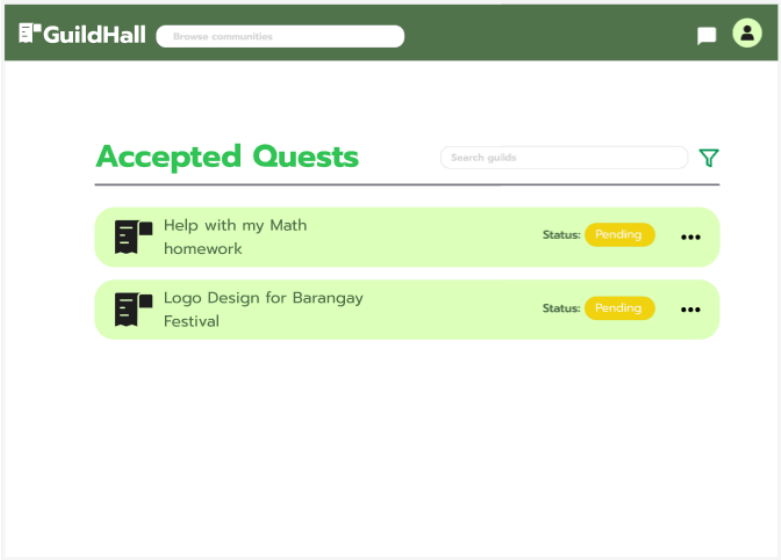




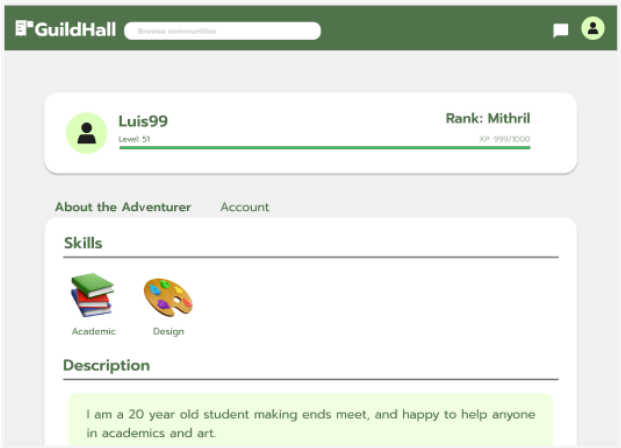
Quest Details

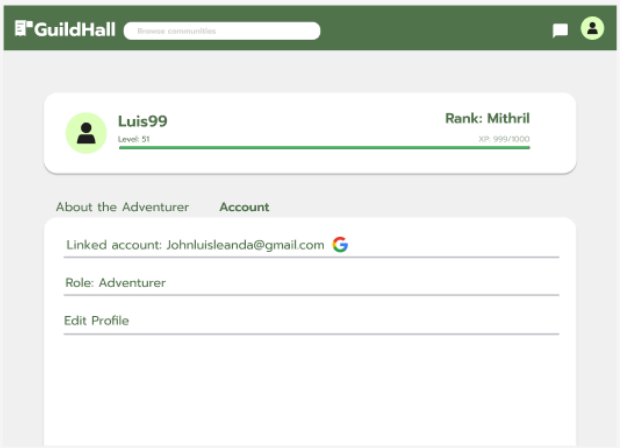


Accepted Quests

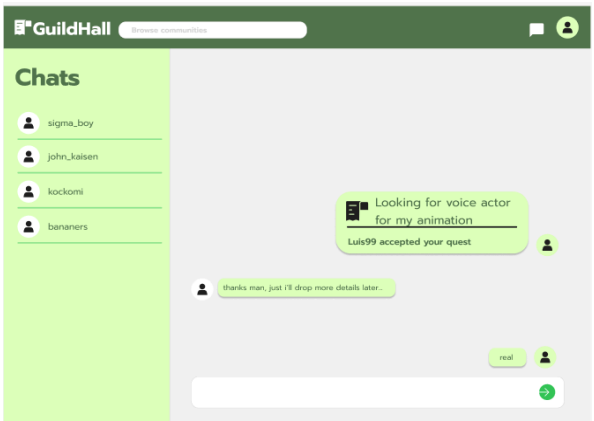


Profile

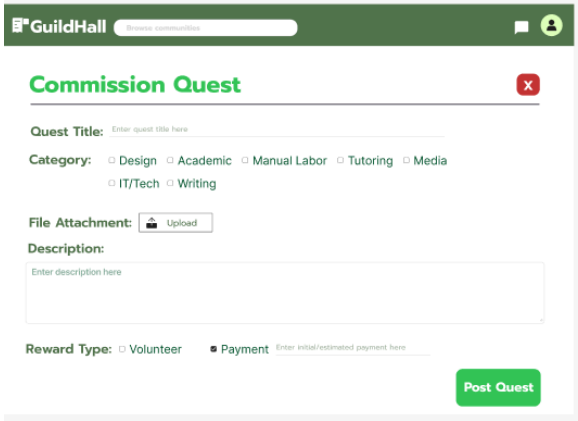


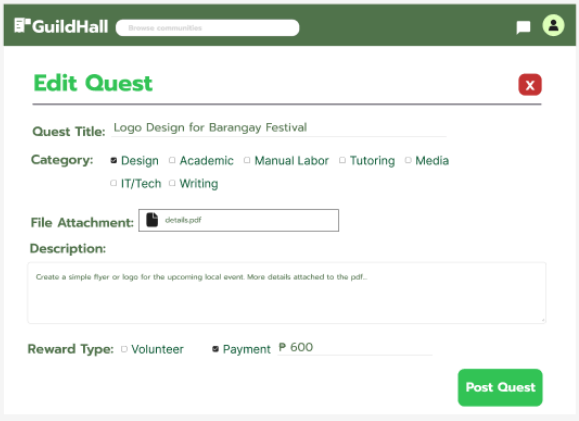


Chat

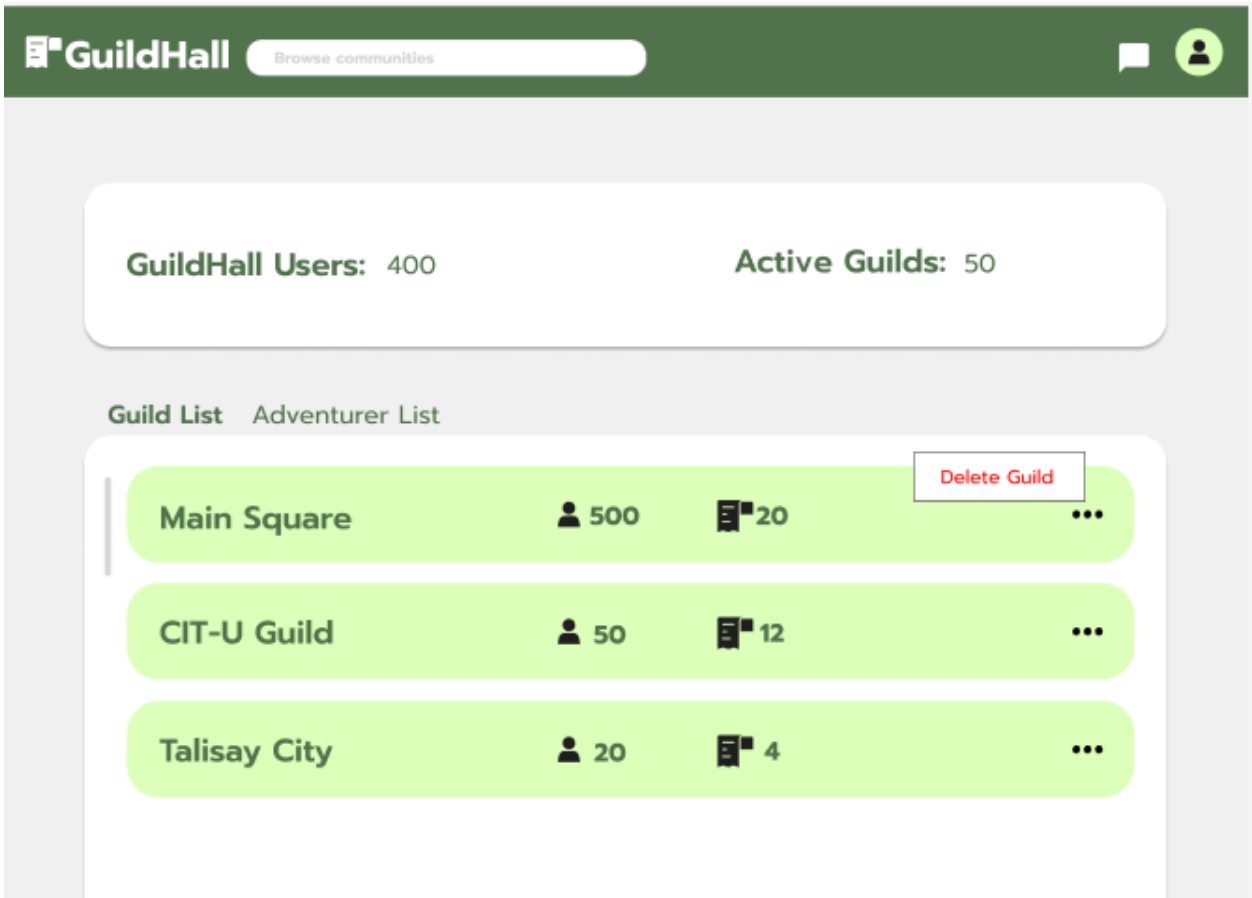


Commission Quest & Edit Quest



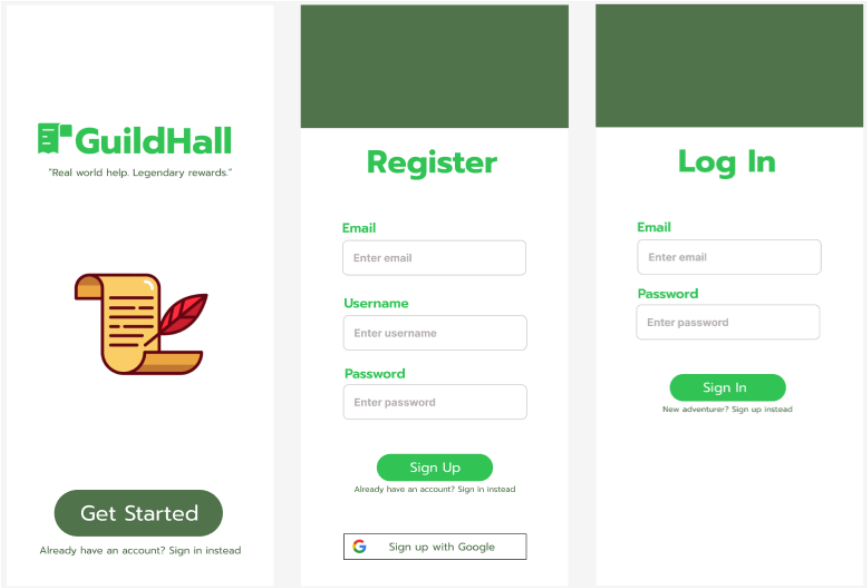


Admin Dashboard

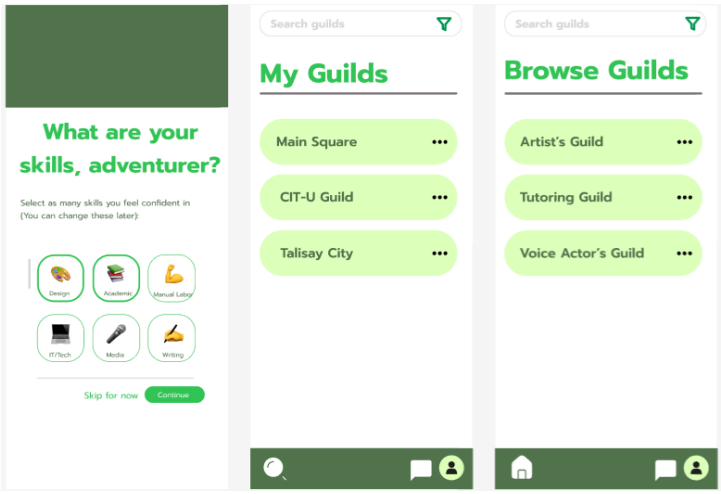


7.2 Mobile Application Wireframes

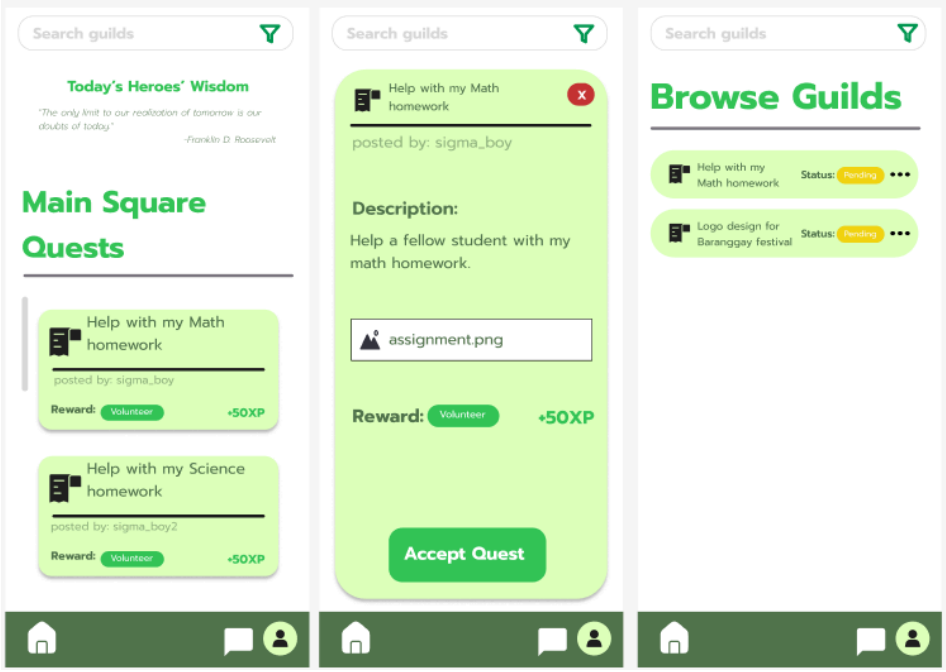
Landing, Register, Login



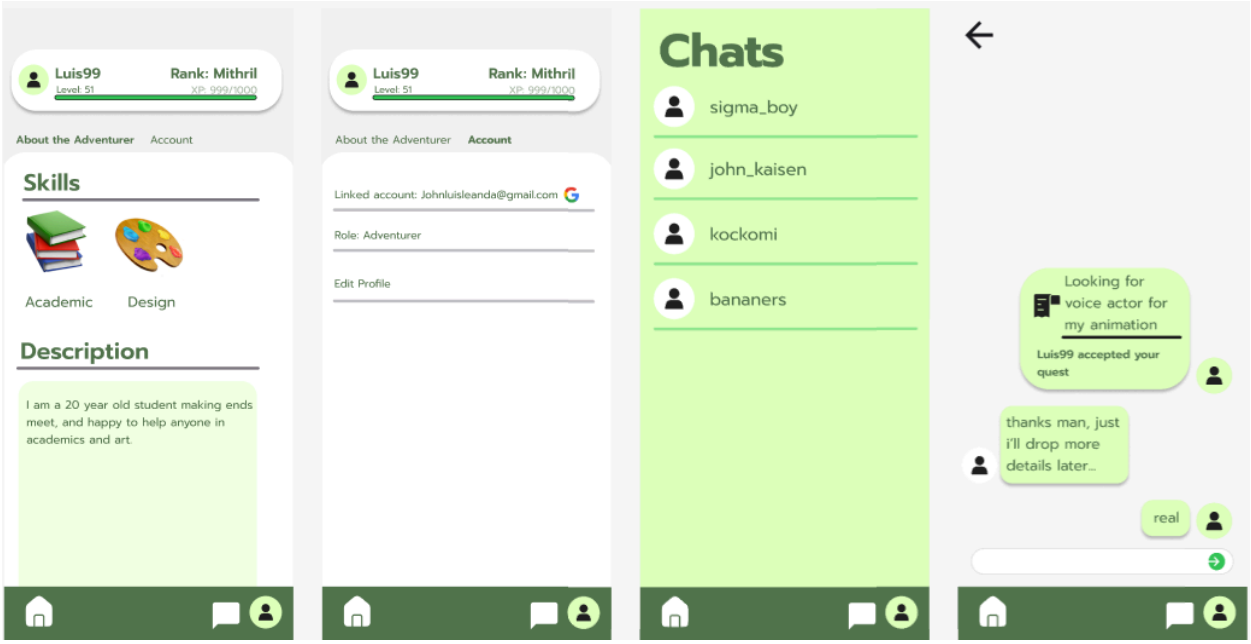
First time Skills selection, Community Page, Browse Communities



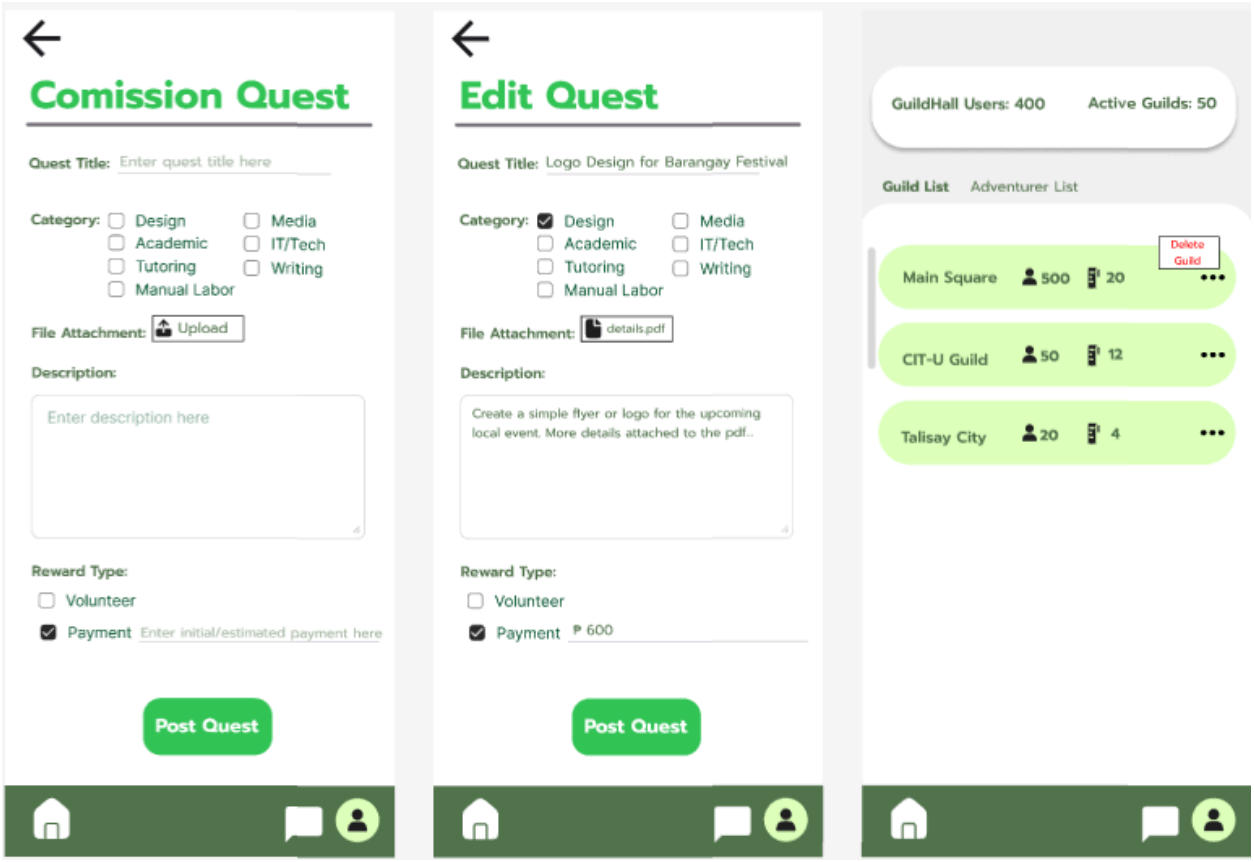
Quest Board, Quest Details, Accepted Quests



Profile, Chat



Commission Quest, Edit Quest, Admin Dashboard



Mobile-Specific Features:

- Touch-optimized buttons (min 48x48px)
- Gesture support (swipe, pull-to-refresh)
- Simplified forms for mobile input

Design System:

- **Colors:** Primary (#34C759), Secondary (#DDFFBC), Tertiary (52734D), Error and Cancel (#C73434), Negative Space (#FFFFFF)
- **Typography:** Font: Prompt
- **Spacing:** 8px grid system
- **Components:** Consistent buttons, inputs, cards, modals
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024p

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and theme finalization (Adventurer/Guild terminology)

Day 3-4: Finalize FRS (Quest Board, Chat, Stripe) and NFRs

Day 5-7: System architecture design (Web, Mobile, Backend, Postgre)

Week 2: Detailed Design

Day 1-2: API specification (REST endpoints for Quests and Guilds)

Day 3-4: Database design (ERD with 6 tables: Users, Guilds, Memberships, Quests, Payments, Messages)

Day 5-6: UI/UX wireframes in Figma (Quest Board, Adventurer Profile, Chat)

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with Security, JPA, and WebSocket dependencies

Day 2: Database configuration and Entity classes

Day 3: JWT Authentication & Google OAuth 2.0 implementation

Day 4: User management & XP/Leveling logic

Day 5: Guild & Membership management

Week 4: Core Features

Day 1: Quest CRUD operations (Create, Browse, Accept)

Day 2: Stripe Payment integration (Bounty Escrow)

Day 3: WebSocket Chat & Automated System Messages

Day 4: Quotable API integration (Heroes' Wisdom) & Error Handling

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup with TypeScript and Tailwind CSS

Day 2: Authentication pages (Google Login, Register)

Day 3: Main Quest Board (Filtering by Guild/Category)

Day 4: Quest Detail Page & Briefing Download

Day 5: Adventurer Profile (XP Bar, Skill Badges)

Week 6: Complete Web Features

Day 1: Real-time Chat interface (WebSocket client)

Day 2: Quest Posting & Stripe Checkout flow

Day 3: Guildmaster Admin Dashboard

Day 4: Responsive design polish

Day 5: API integration and end-to-end testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup & XML Layout structures

Day 2: Authentication Screens

Day 3: Quest Browsing

Day 4: API Service Layer

Day 5: Briefing Download & File Handling

Week 8: Complete Mobile App

Day 1: Active Quest Management & Accept logic

Day 2: Mobile Chat implementation

Day 3: UI polish

Day 4: Testing on physical Android device/emulator

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: Cross-platform testing

Day 2: Bug fixes (specifically Chat and Stripe webhooks)

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation finalization

Week 10: Deployment

Day 1: Web app deployment (Render)

Day 2: Mobile APK distribution (GitHub Release)

Day 3: Final system health check

Day 4: Project submission & Presentation prep

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Quest Status State Machine (Week 4)
3. Real-time WebSocket Sync (Week 6-8)
4. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements
- Test Stripe and WebSockets in isolation before adding them to the main UI
- Focus on the "Global Square" first before adding private Guilds